Sonder — 12-Minute Group Script

Format: ~7 minutes of speaking + ~5 minutes of live demo + Q&A. Group of four (Shreyas, Mushahid, Ali, Jahnvi).

Speaker tags in `[BRACKETS]` at the start of each section. Times are *target* — under is fine, over is not.

Open (~20 sec) — [SHREYAS]

Sonder. Travel, together.

We're four people who built one product that does three things together — none of which exist anywhere else in one place.

In twelve minutes we'll walk you through the system, then show it running. Let's go.

Advance.

Problem (~60 sec) — [SHREYAS]

Travel today is fragmented across four broken surfaces. TripAdvisor ranks restaurants context-free. Google Travel hands you a hotel and walks away. Instagram inspires you but a saved reel isn't an itinerary. Airbnb books you a room.

You end up with five tabs open and a Notes doc.

The harder problem nobody is solving — **who do you travel with.** Solo travellers get nothing. Couples get hotel suggestions. Anyone wanting to share a trip with a compatible stranger falls back to hostel bars and dating apps.

That's the gap we sat in. Not "better recommendations" — but the absence of a system that knows your rhythm, plans around it, and finds people whose rhythm matches.

Advance.

What Sonder is (~50 sec) — [SHREYAS]

Three things together. Nowhere else.

One — real-venue itineraries grounded in actual places that exist. We pull the activity corpus from the Foursquare Places API. The LLM never invents a venue.

Two — compatibility matching that takes the actual signal seriously. Twelve push-pull motivation dimensions, an eight-key emotional signature derived from a GoEmotions cosine classifier, a multi-feature ranker that adjusts to your feedback after every revision.

Three — a believable social layer alive on day one. 192 LLM-designed personas plus 18 couples, autonomously posting, opening trips, and reaching out to real users.

Advance.

Architecture, four planes (~70 sec) — [MUSHAHID]

The system is four service planes operating independently.

Cloudflare Pages serves the React frontend from 300+ edge points of presence. A catch-all Pages Function proxies every `/api/*` request to the backend so the SPA stays on a single domain — required for service-worker push registration.

Render runs the FastAPI backend. Single long-lived container with WebSocket support, SSE streaming for trip generation, and a background loop for the synthetic agents.

Pinecone is the vector index — 50,000 vectors across four namespaces: destinations, activities, restaurants, hotels, and co-travellers. Co-travellers live in the same index so retrieval is a single API call.

Firestore holds operational state — user profiles, itineraries, chats, shared itineraries, social posts, push subscriptions.

The point of separating them is *operational independence at scale.* Render can restart and search vectors stay live. Pinecone can be slow and the dashboard still loads from Firestore.

Advance.

AI stack + RAG pipeline (~75 sec) — [ALI]

Every LLM call routes through a tier classifier.

Small tier for chat, openers, classifiers, RAG explanations — Claude Haiku 4.5 primary, GPT-4o-mini fallback.

Large tier for itinerary generation and complex refinement — Claude Sonnet 4.6 primary, GPT-4o fallback.

Validators are explicitly a *different provider family* from the generator — NVIDIA NIM running Nemotron-Nano-8B on the small validator, GPT-5-mini on the large. Cross-provider critique to avoid same-family blind spots.

When the user hits "plan my trip," seven stages run. Three persona-inference pipelines in parallel, destination retrieval, activity retrieval, ranking, then the large-tier LLM streams the itinerary day-by-day. The frontend's JSON parser reads each closing brace and shows Day 1 within fifteen seconds while Day 7 is still being written.

The validator runs on the finished trip. Issues trigger a refinement loop. The user only sees the trip that passed.

Advance.

Validator engine (~70 sec) — [MUSHAHID]

This is the piece we're most proud of architecturally.

Every LLM call is graded by a second LLM, from a different provider family, against a surface-specific rubric. Five rubrics — itinerary, persona reveal, co-traveller match, chat reply, chat-reply repair.

A deterministic regex pre-check runs first. Instant, no LLM cost. Catches cheap failures before they reach a model.

The piece that matters operationally is **async edit-in-place repair.** Chat reply broadcasts immediately. Validator runs in the background. If it catches something — a Paris persona accidentally referencing Lisbon on a Japan trip — we write the corrected reply and swap it in via a `message_edited` WebSocket event. User reads the original for half a second, then the corrected version slides in.

Large-model-validated quality at small-model latency.

Advance.

What makes it work (~70 sec) — [JAHNVI]

Two architectural decisions running through everything.

One — information starvation as quality control. The persona inferrer never sees the embedding vector. The validator never sees the user's original prompt. The matcher's policy file never sees individual user data. Each stage knows exactly what it needs to do its job, and *nothing else.* Forced specialisation produces better output.

Two — the two-stage blind-writer pipeline for synthetic personas. Stage one receives only raw signal — city, age, gender, four persona-question option keys — and writes the character. Stage two reads the character output and assigns the motivation profile + emotional signature, blind to the original keys.

A language model with the answer key in front of it writes to that key. A language model that has to guess writes something honest.

That's how 192 synthetic travellers feel like real people the moment a user lands on the dashboard.

Advance.

Demo intro (~15 sec) — [SHREYAS]

Let's show it.

Advance to demo. Jahnvi drives.

DEMO (~5 minutes) — [JAHNVI drives, team narrates]

Suggested flow (don't read aloud — these are stage directions):

1. **(40s)** Land on the empty dashboard. Show the concierge banner, the rotating destination cycle, the spinning 3D Sonder mark. Set tone — *"this is the empty state. No trips yet."* 2. **(60s)** Click *Plan a trip.* Walk through preferences — destination, dates, persona questions. Hit reveal. 3. **(60s)** Cinematic destination reveal lands. Show the streamed itinerary appearing day-by-day. Open one activity card — show the "Why this?" explanation. 4. **(45s)** Lock in. Open the matches column. Click a curated traveller. Brief look at the match-detail page — compatibility breakdown, persona snapshot. 5. **(45s)** Start the vibe-check chat. Send one message. Receive the persona's reply *(callout: that came from Claude Haiku 4.5, validated by NVIDIA Nemotron in the background).* 6. **(30s)** Open the shared-itinerary surface. Show a proposed-change in mid-negotiation. Pull the curtain back briefly. 7. **(30s)** Pulse / Discover — surface the synthetic personas operating autonomously. Show the live-traveller strip.

If a step lags, skip it. Land on Pulse so the audience sees the system *alive* with synthetic activity.

Close (~25 sec) — [SHREYAS]

Sonder.

Beat.

A travel system that knows you, plans around you, and finds people who match you.

Beat.

Repo's on screen. We'll take questions.

Stop talking.

Speaker order summary

Time	Section	Speaker
0:00	Open	Shreyas
0:20	Problem	Shreyas
1:20	What Sonder is	Shreyas
2:10	Architecture — four planes	Mushahid
3:20	AI stack + RAG pipeline	Ali
4:35	Validator engine	Mushahid
5:45	What makes it work	Jahnvi
6:55	Demo intro	Shreyas
7:10	DEMO	Jahnvi drives
12:10	Close	Shreyas

Buffer is ~50 seconds across the whole talk. Don't burn it on slide 1.

Delivery notes

- **Pace.** Spoken English at a slide-presentation pace is ~120 words per minute. Each section's target time was calculated against that. If a section feels long when you rehearse, *cut*, don't speed-read.
- **Hand-offs are silent.** No "and now I'll pass it to…" — the next speaker just starts. Practise three times so it lands clean.
- **The strongest line in the deck:** *"A language model with the answer key in front of it writes to that key. A language model that has to guess writes something honest."* — that's Jahnvi's. Let it breathe.
- **The strongest operational line:** *"Large-model-validated quality at small-model latency."* — that's Mushahid's. Let it land.
- **Demo failure protocol:** if anything 500s during the live demo, Jahnvi keeps narrating, Shreyas pulls up the recorded backup. No silence.